Data Network
Final Project

Iman Alizadeh Fakouri

Student Number: 401102134

Instructor: Dr. Pakravan

Date: February 26, 2026

Contents

1 DNS Protocol **3**
1.1 Questions 3
1.2 Tasks 3

2 FTP Protocol **9**
2.1 RETR 9
2.2 STOR 13

3 Bonus Part: Integration **16**

DNS Protocol

1.1 Questions

a.

The 16-bit flag field directs how a DNS message is handled. QR distinguishes between a query and a response. Opcode identifies the query type. AA flags an authoritative answer. TC indicates if the message was truncated. RD is set by the client to request recursive resolution. RA is set by the server to confirm recursion is available. Z is reserved and kept at zero. RCODE provides the outcome of the query, such as success or non-existent domain.

b.

DNS utilizes UDP over port 53. UDP is connectionless and avoids the three-way handshake required by TCP, allowing queries to be processed significantly faster and minimizing overhead. TCP is only used as a fallback when responses exceed the length limit or for reliable administrative tasks like zone transfers.

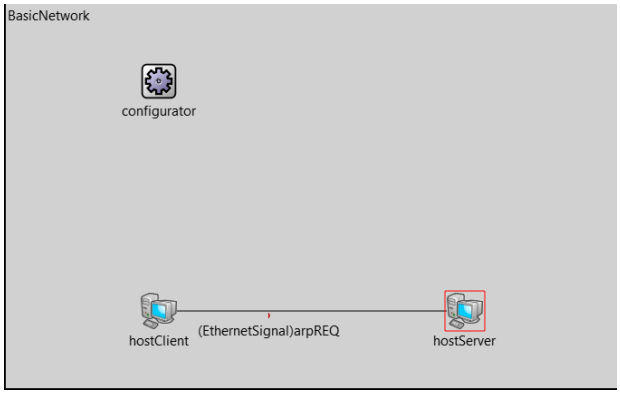
c.

DNS caching drastically improves performance by storing recently resolved IP addresses locally, which reduces resolution latency and network traffic. The Time to Live value determines exactly how long a record stays in the cache. A short Time to Live ensures rapid updates for changing IPs but generates more query traffic, while a long Time to Live maximizes cache efficiency.

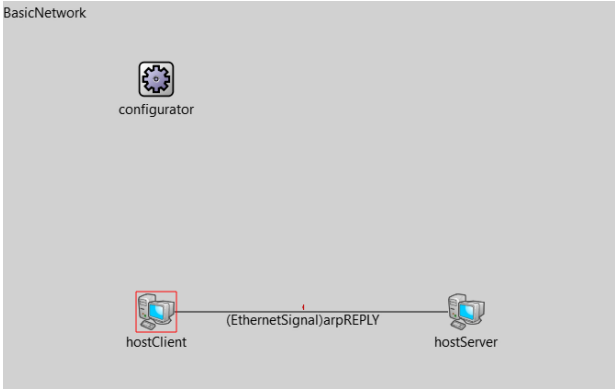
1.2 Tasks

a. Basic DNS Resolution

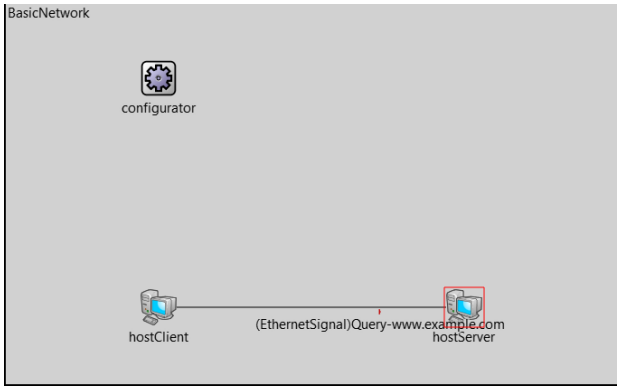
A custom DNS application was implemented within the INET framework using socket programming. The client queries the server, which resolves the domain using a local XML database and replies with the mapped IP address and a specific Time to Live value. The client caches this result and initiates an expiration timer. During the simulation, the nodes exchange ARP packets to discover MAC addresses. Following this, the client sends the initial DNS query and receives an answer containing a 5-second Time to Live. Exactly 5 seconds later, the cache expires, and the client automatically generates a fresh query.



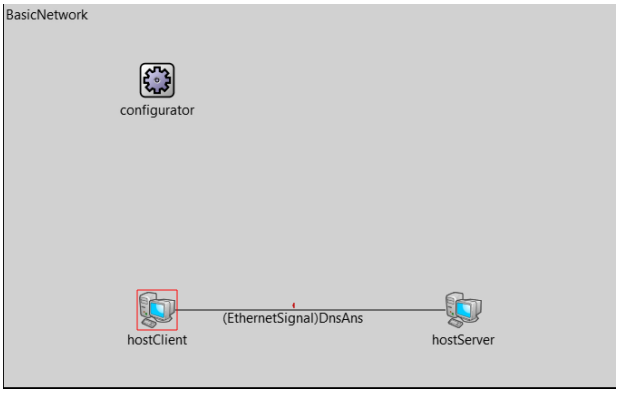
(a) ARP Request



(b) ARP Reply



(c) DNS Query



(d) DNS Answer

Figure 1: OMNeT++ simulation message exchange sequence

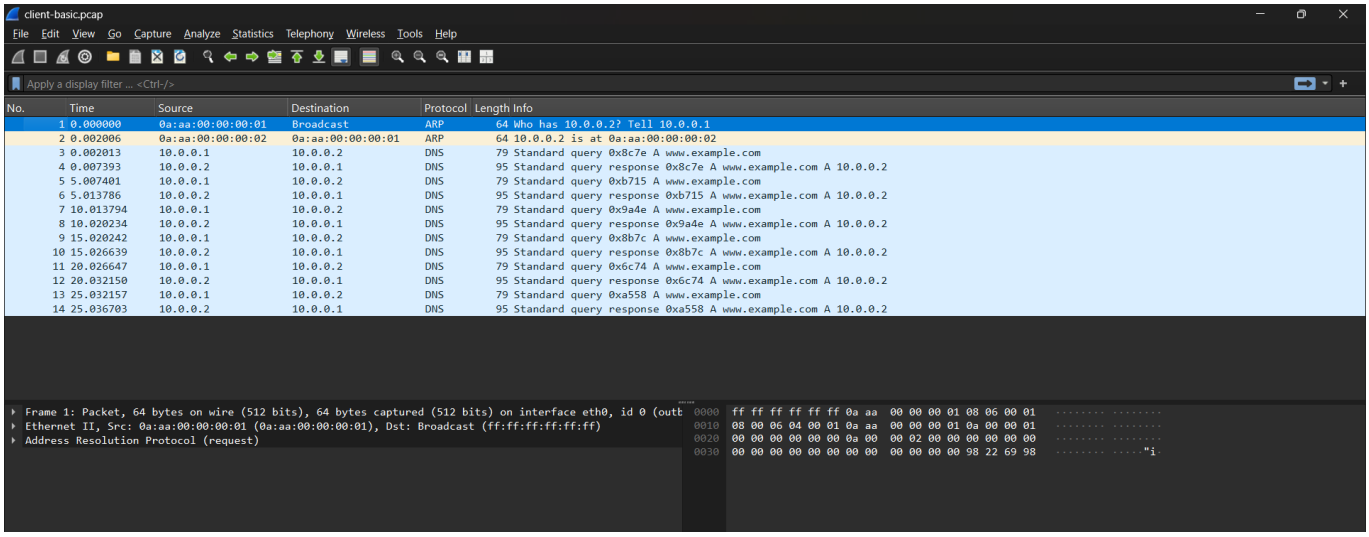


Figure 2: Wireshark capture showing 5-second query intervals based on expiration

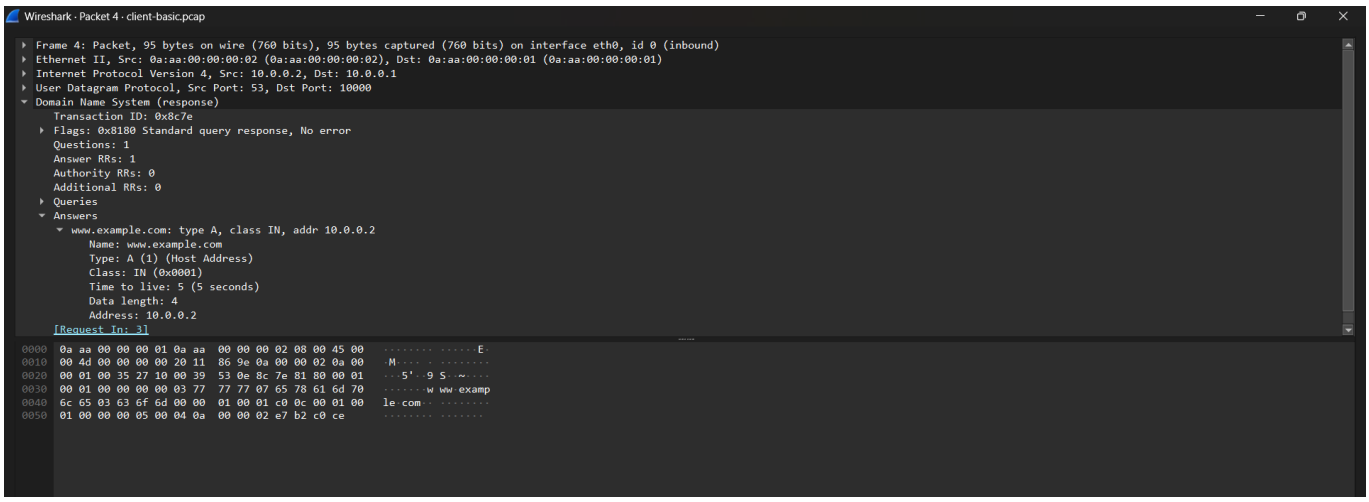


Figure 3: Packet details displaying the provided TTL value of 5 seconds

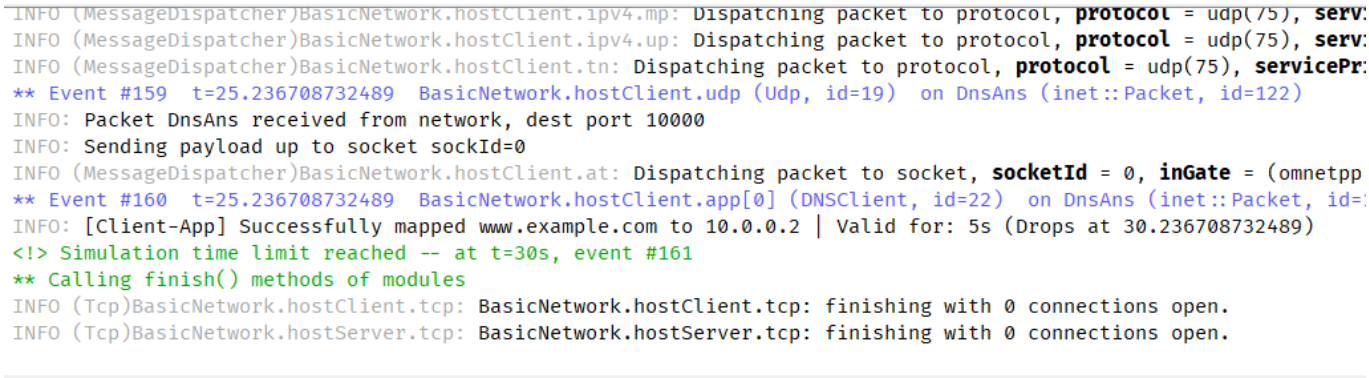
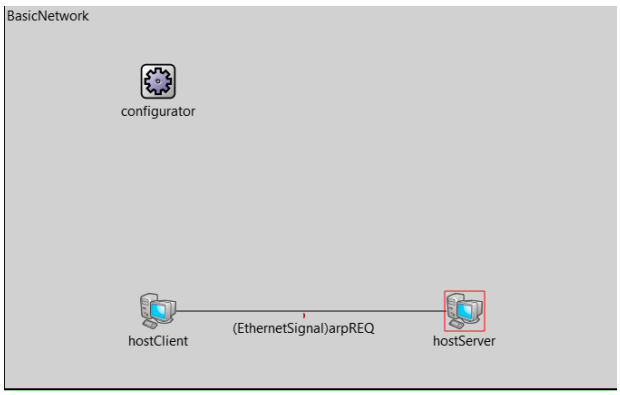


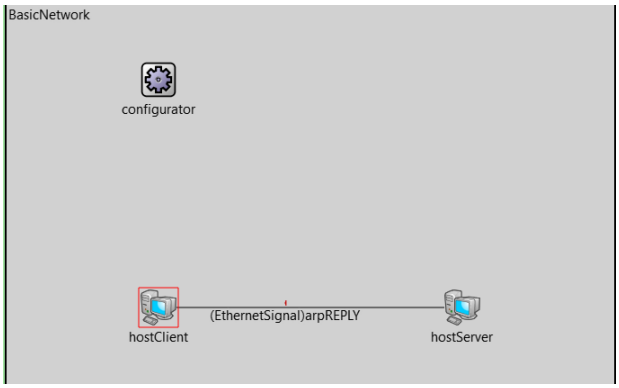
Figure 4: Console log confirming application mapping and cache expiration behavior

b. DNS Resolution with Non-Existent Record

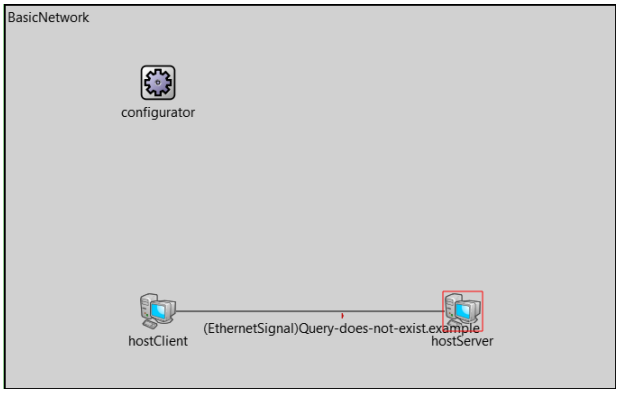
In this scenario, the client queries the server for a domain that does not exist in the local XML database. After the initial ARP address resolution, the client sends the DNS query. The server searches its database, fails to find a matching record, and returns a DNS error response indicating a non-existent domain. The client correctly interprets this error code, aborts the caching process, and logs the resolution failure. Because no record is cached, no refresh timer is triggered, and the query is not repeated.



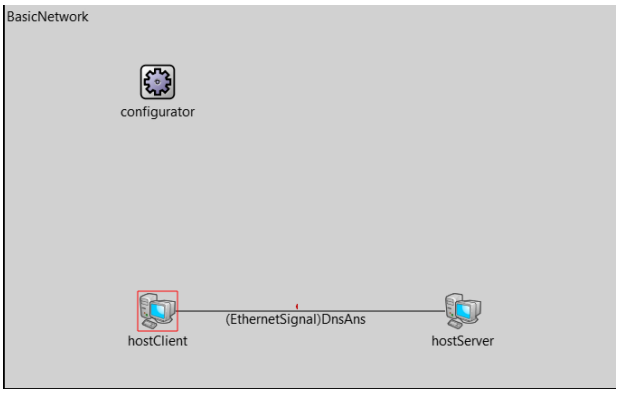
(a) ARP Request



(b) ARP Reply



(c) DNS Query for non-existent domain



(d) DNS Answer with error

Figure 5: OMNeT++ simulation message exchange sequence for non-existent record

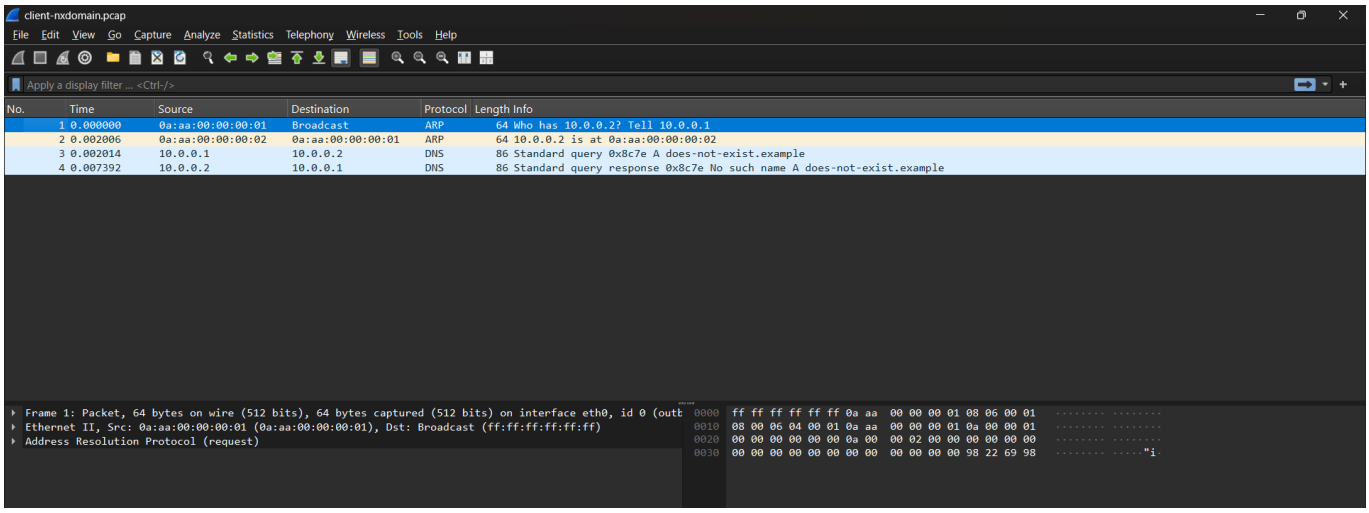


Figure 6: Wireshark capture showing single query and error response with no subsequent retries

Figure 7: Packet details displaying the No such name reply code

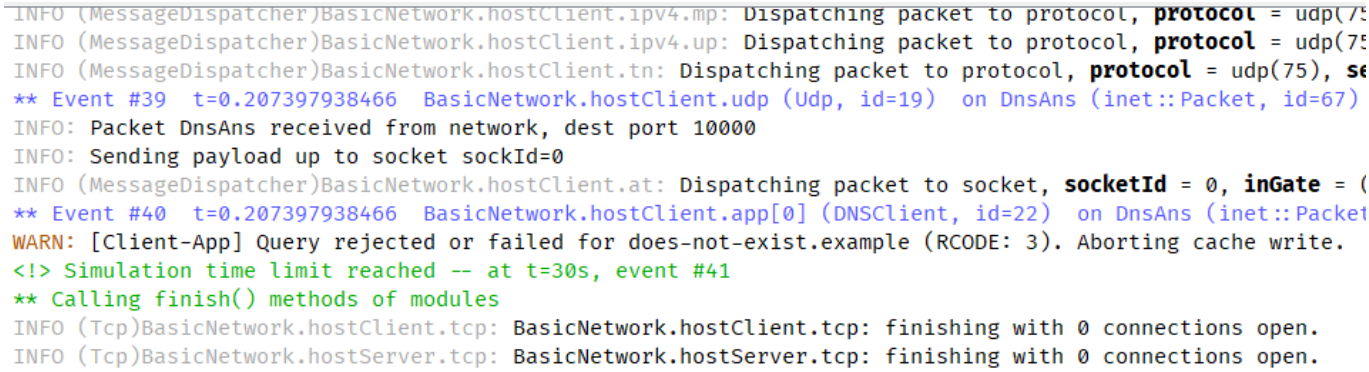


Figure 8: Console log confirming the client aborting the cache write

c. Hierarchical DNS Resolution

In this scenario, a hierarchical DNS architecture was implemented using a client, a local DNS server, and a root DNS server. The local server acts as a proxy with caching enabled, while the root server acts as the authoritative source providing a 20-second Time to Live for the records. To evaluate the caching performance, the client is configured to issue an initial query and a subsequent query roughly 5 seconds later, well before the cache expires.

The packet captures clearly illustrate the caching mechanism and its impact on performance. The client capture demonstrates that the initial query experiences a resolution delay of approximately 23 milliseconds because it must traverse the network to the root server. In contrast, the second query sent near the 5-second mark is resolved in just 5 milliseconds. The local DNS capture confirms this logic by showing the first request being forwarded to the root server, while the second request is answered directly from the local cache. Finally, the root DNS capture proves that the top-level server is completely bypassed during the second query, as no traffic is recorded at the 5-second mark.

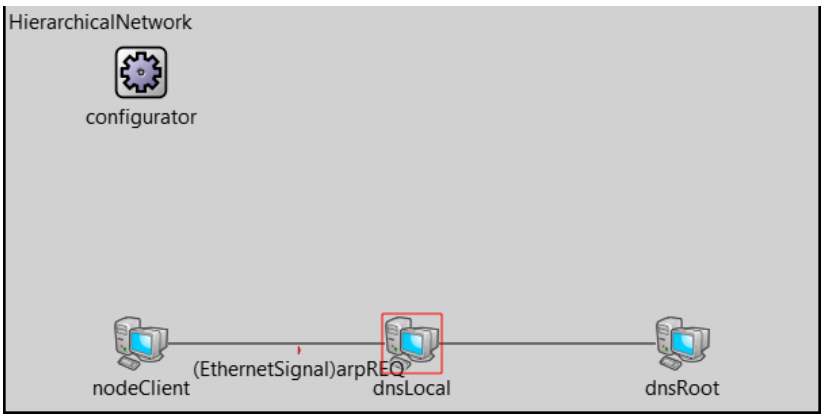


Figure 9: OMNeT++ simulation topology for hierarchical DNS resolution

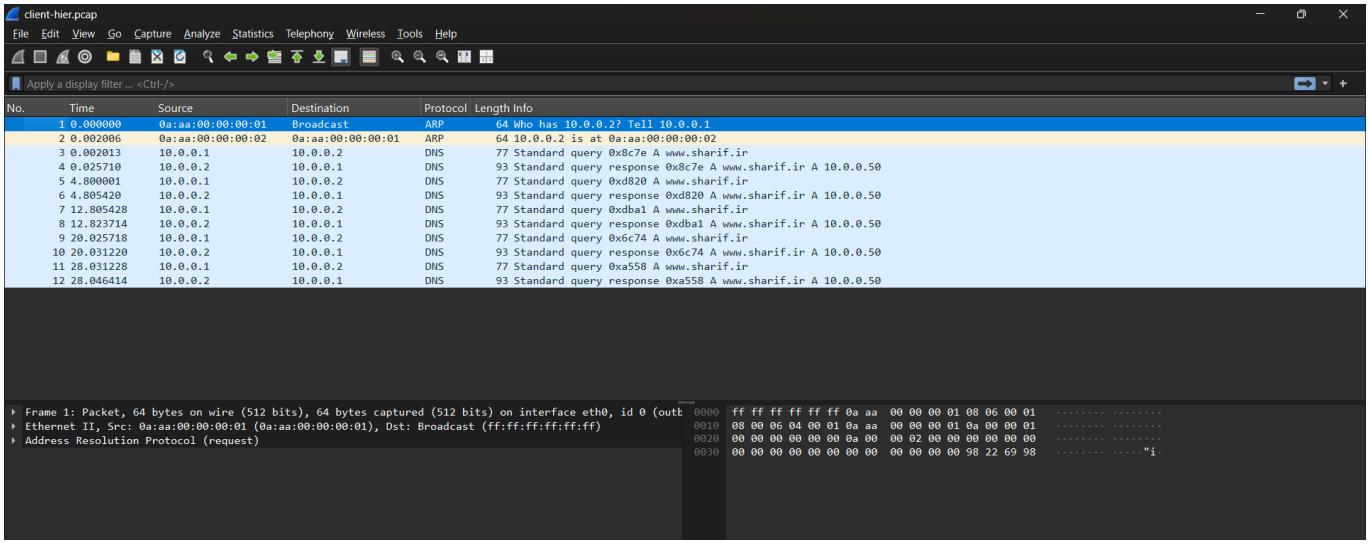


Figure 10: Client packet capture highlighting the difference in resolution delay between the non-cached and cached queries

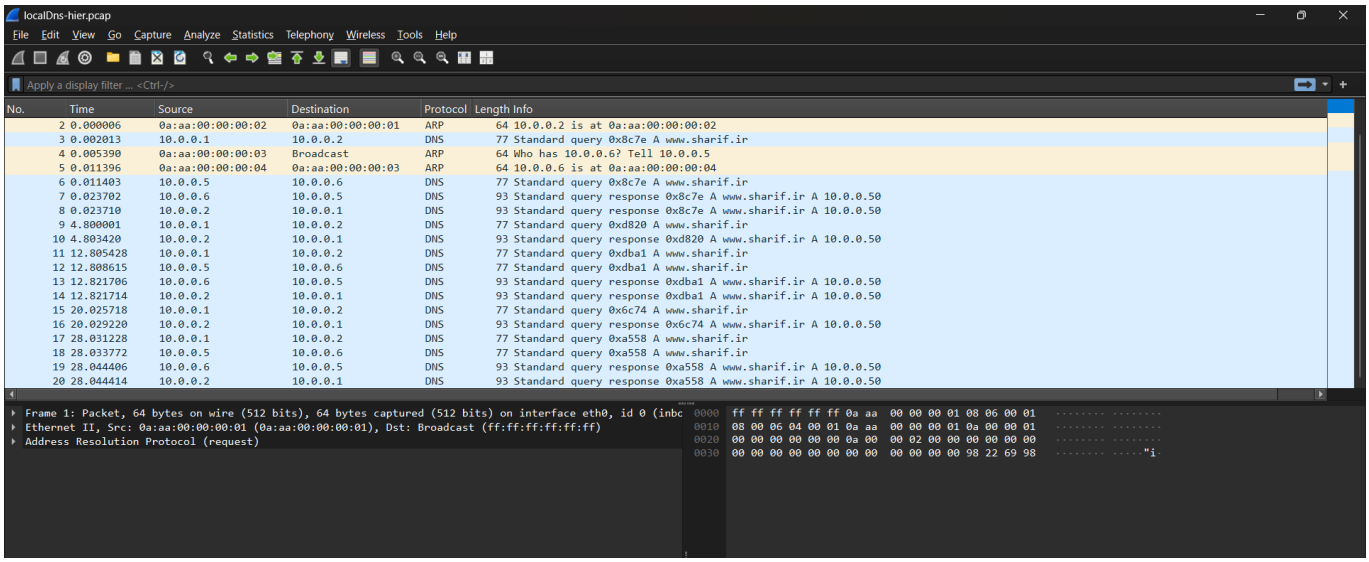


Figure 11: Local DNS packet capture showing proxying behavior for the first query and direct cache fulfillment for the second query

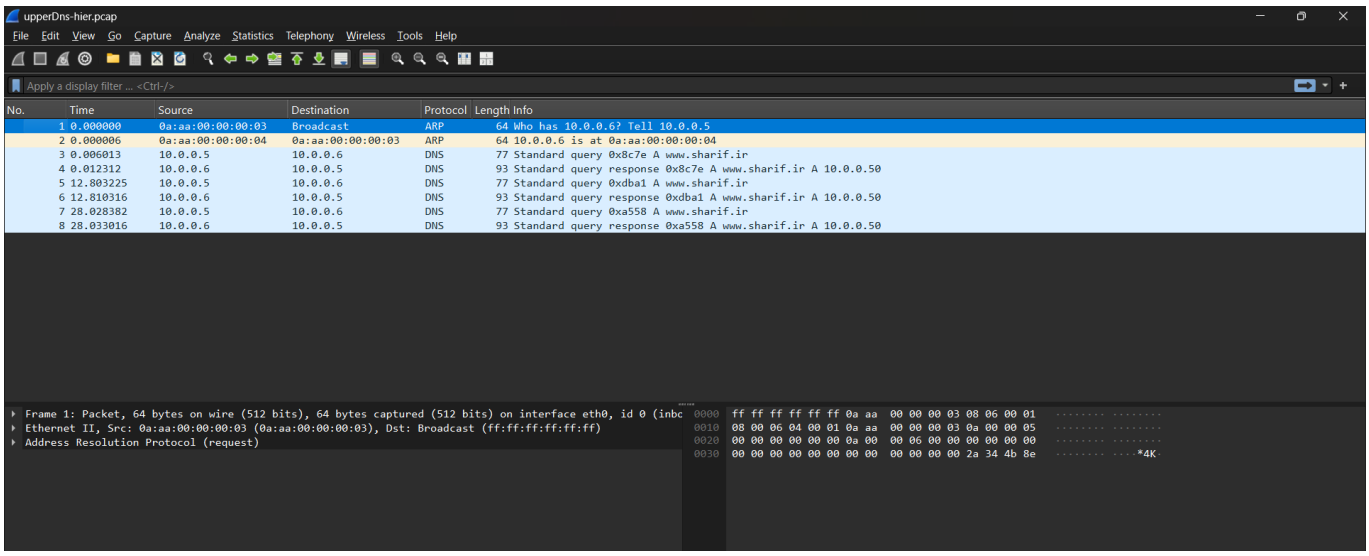


Figure 12: Root DNS packet capture confirming it only receives traffic during a cache miss at the local level

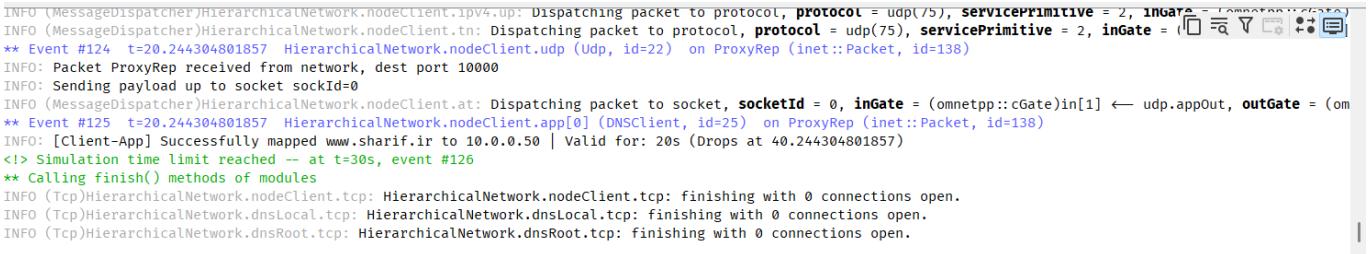


Figure 13: Console log confirming successful resolution and assignment of the 20-second Time to Live from the root server

---

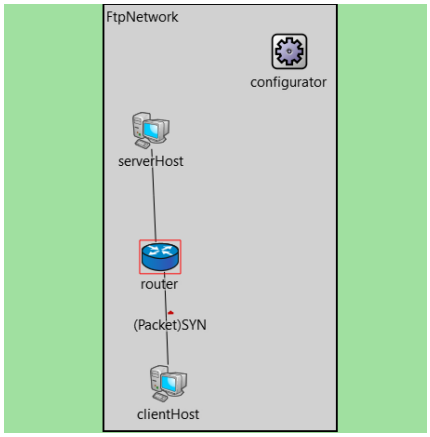
FTP Protocol

2.1 RETR

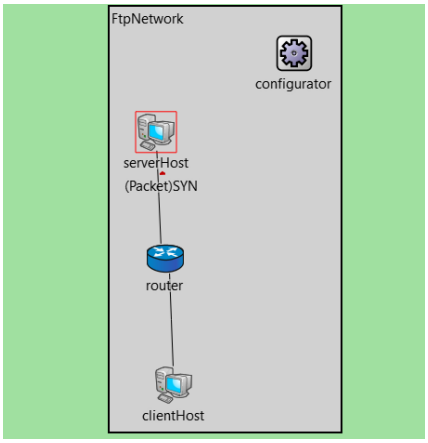
The File Transfer Protocol (FTP) implementation follows a dual-socket architecture, utilizing a persistent control channel (Port 21) for commands and a dynamic data channel for file transfers. In this scenario, the client requests a file download using the `RETR` command.

Simulation Visuals

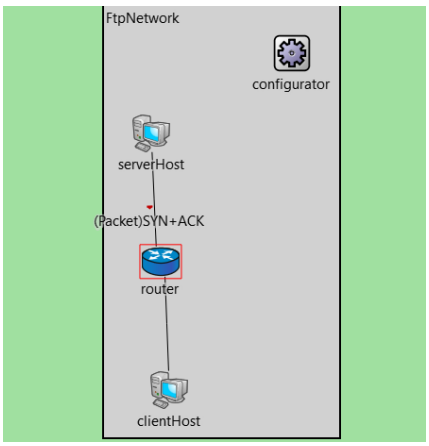
The following screenshots capture the chronological progression of packets through the `FtpNetwork` topology, showing the interaction between the clientHost, router, and serverHost.



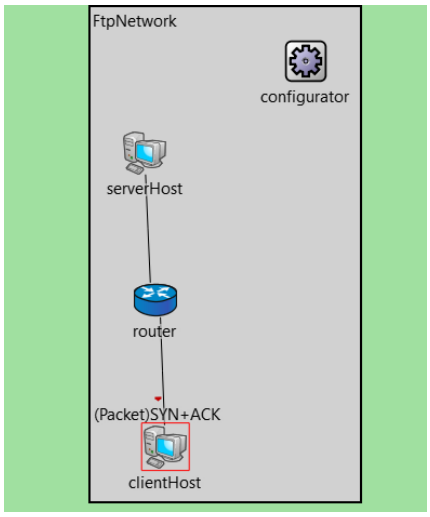
(a) SYN: Client to Router



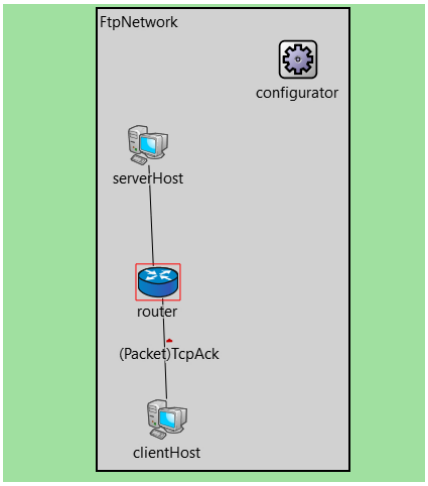
(b) SYN: Router to Server



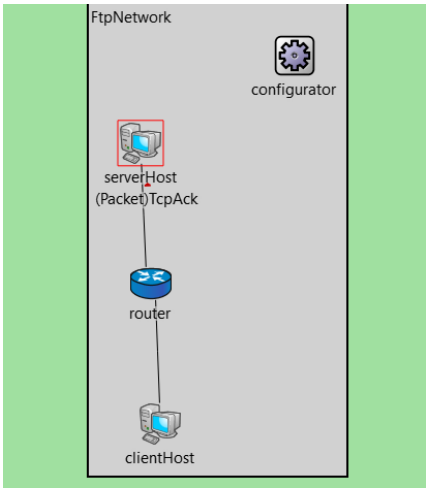
(c) SYN+ACK: Server to Router



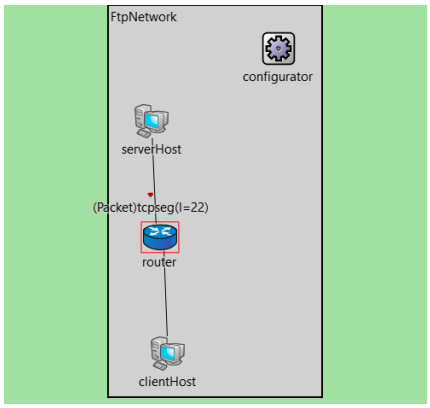
(d) SYN+ACK: Router to Client



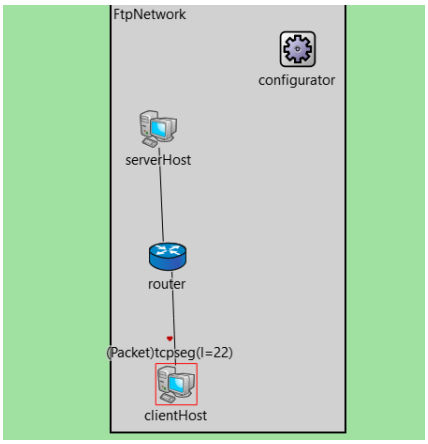
(e) TCP Ack: Client to Router



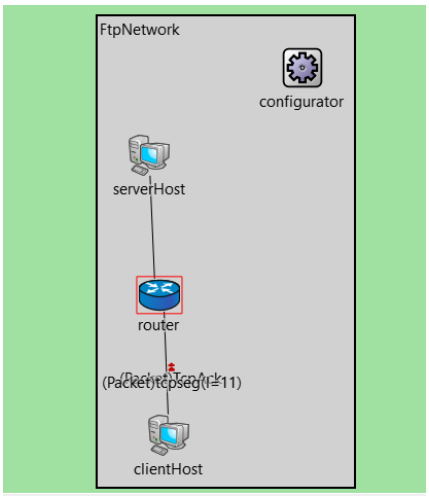
(f) TCP Ack: Router to Server



(g) tcpseg (l=22): Server to Router



(h) tcpseg (l=22): Router to Client



(i) tcpseg (l=11): Client to Router

Figure 14: Chronological packet propagation through the network nodes

General Operation and Connection Model

The process begins with the client establishing a TCP control connection to the server's Port 21. After authentication via the `USER` command, the client enters "Active Mode". It starts listening on

a local port (2020) and informs the server of this address and port using the `PORT` command. Upon receiving a valid `RETR data.txt` command, the server initiates a secondary TCP connection from its side to the client's specified listening port. The file content, which in this test contained the string `"I am Iman Fakury."`, is transmitted over this dedicated data channel. Once the transfer is complete, the data channel is closed, and the server sends a confirmation (226) over the control channel.

Wireshark and Log Analysis

The following captures provide a microscopic view of the protocol's state machine and the underlying TCP mechanics verifying the dual-channel architecture.

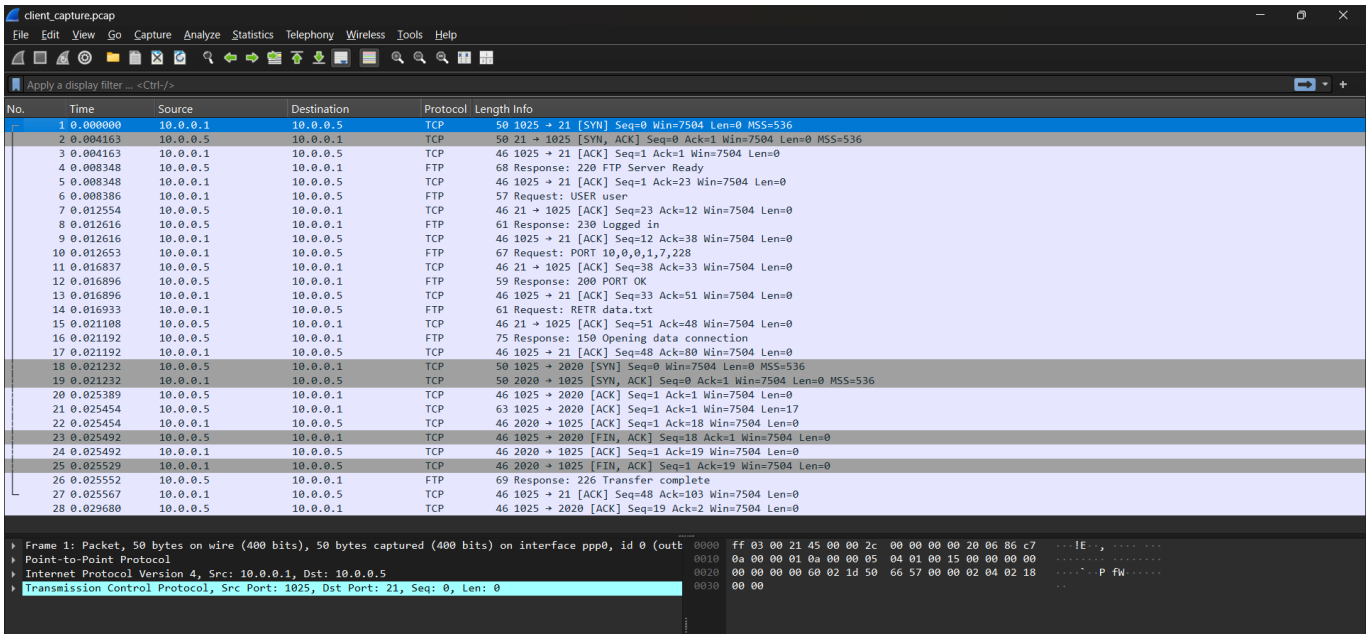
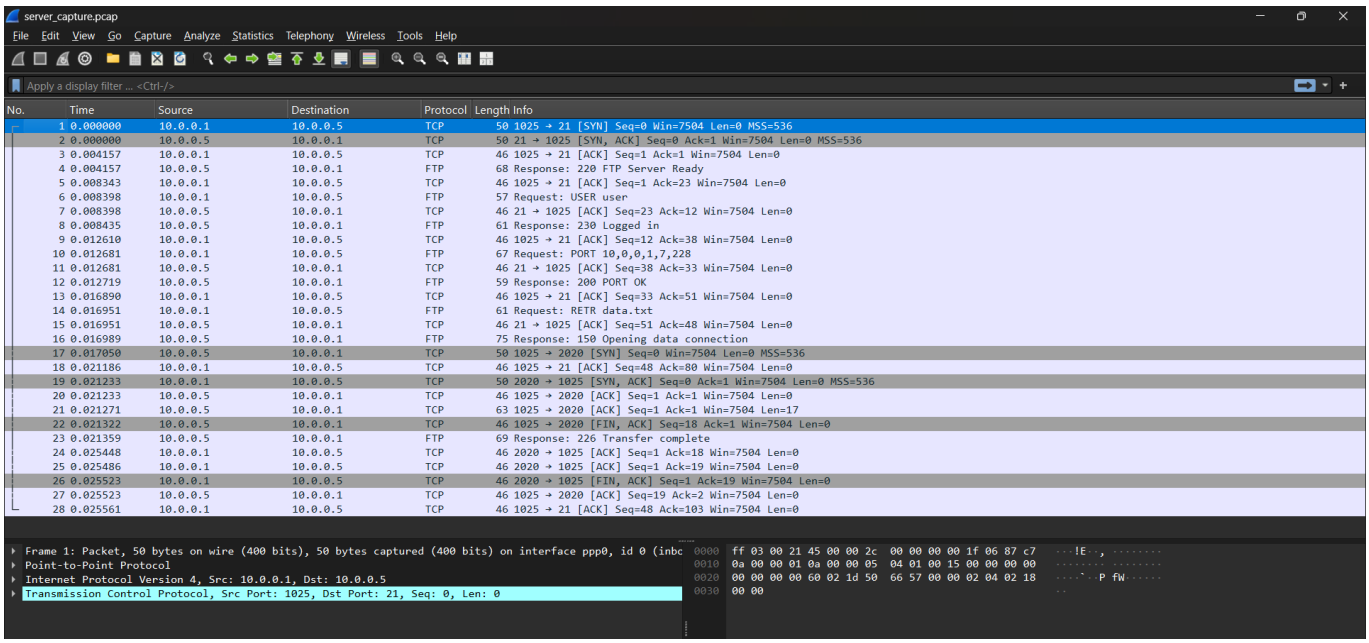


Figure 15: Client-side PCAP: Exhibits the persistent control channel on Port 21 and the server initiating the data connection on Port 2020

The client-side packet capture chronologically maps the dual-port architecture. Packets 1 through 17 demonstrate the control plane exchange, validating the sequential progression of the `USER`, `PORT`, and `RETR` commands over port 21. Crucially, starting at packet 18, the capture records an incoming TCP SYN segment directed to the client's locally bound port 2020. This explicitly verifies that the client successfully transitioned into a listening state and that the server actively routed the data channel connection back to the client's specified network coordinates.

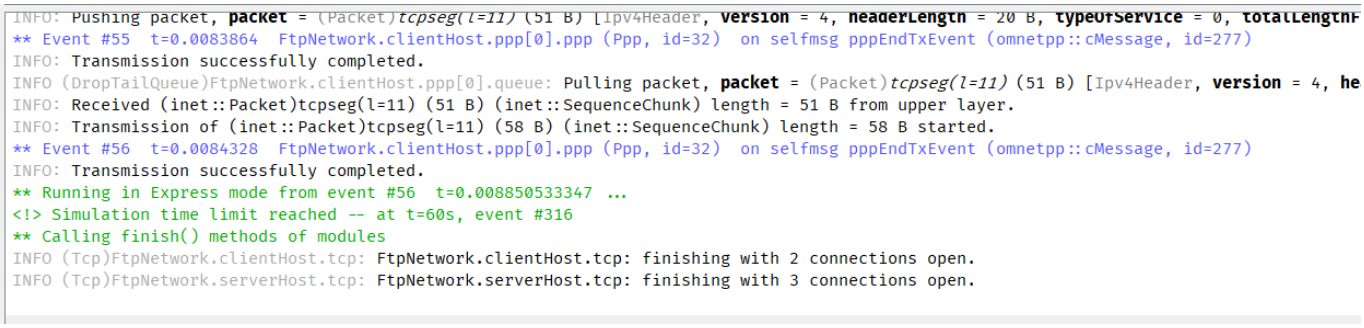


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.1	10.0.0.5	TCP	50	1025 → 21 [SYN] Seq=0 Win=7504 Len=0 MSS=536
2	0.000000	10.0.0.5	10.0.0.1	TCP	50	21 → 1025 [SYN, ACK] Seq=0 Ack=1 Win=7504 Len=0 MSS=536
3	0.004157	10.0.0.1	10.0.0.5	TCP	46	1025 → 21 [ACK] Seq=1 Ack=1 Win=7504 Len=0
4	0.004157	10.0.0.5	10.0.0.1	FTP	68	Response: 220 FTP Server Ready
5	0.008343	10.0.0.1	10.0.0.5	TCP	46	1025 → 21 [ACK] Seq=1 Ack=23 Win=7504 Len=0
6	0.008398	10.0.0.1	10.0.0.5	FTP	57	Request: USER user
7	0.008398	10.0.0.5	10.0.0.1	TCP	46	21 → 1025 [ACK] Seq=23 Ack=12 Win=7504 Len=0
8	0.008435	10.0.0.5	10.0.0.1	FTP	61	Response: 230 Logged in
9	0.012610	10.0.0.1	10.0.0.5	TCP	46	1025 → 21 [ACK] Seq=12 Ack=38 Win=7504 Len=0
10	0.012681	10.0.0.1	10.0.0.5	FTP	67	Request: PORT 10,0,0,1,7,228
11	0.012681	10.0.0.5	10.0.0.1	TCP	46	21 → 1025 [ACK] Seq=38 Ack=33 Win=7504 Len=0
12	0.012719	10.0.0.5	10.0.0.1	FTP	59	Response: 200 PORT OK
13	0.016890	10.0.0.1	10.0.0.5	TCP	46	1025 → 21 [ACK] Seq=33 Ack=51 Win=7504 Len=0
14	0.016951	10.0.0.1	10.0.0.5	FTP	61	Request: RETR data.txt
15	0.016951	10.0.0.5	10.0.0.1	TCP	46	21 → 1025 [ACK] Seq=51 Ack=48 Win=7504 Len=0
16	0.016989	10.0.0.5	10.0.0.1	FTP	75	Response: 150 Opening data connection
17	0.017050	10.0.0.5	10.0.0.1	TCP	50	1025 → 2020 [SYN] Seq=0 Win=7504 Len=0 MSS=536
18	0.021186	10.0.0.1	10.0.0.5	TCP	46	1025 → 21 [ACK] Seq=48 Ack=80 Win=7504 Len=0
19	0.021233	10.0.0.1	10.0.0.5	TCP	50	2020 → 1025 [SYN, ACK] Seq=0 Ack=1 Win=7504 Len=0 MSS=536
20	0.021233	10.0.0.5	10.0.0.1	TCP	46	1025 → 2020 [ACK] Seq=1 Ack=1 Win=7504 Len=0
21	0.021271	10.0.0.5	10.0.0.1	TCP	63	1025 → 2020 [ACK] Seq=1 Ack=1 Win=7504 Len=17
22	0.021322	10.0.0.5	10.0.0.1	TCP	46	1025 → 2020 [FIN, ACK] Seq=18 Ack=1 Win=7504 Len=0
23	0.021359	10.0.0.5	10.0.0.1	FTP	69	Response: 226 Transfer complete
24	0.025448	10.0.0.1	10.0.0.5	TCP	46	2020 → 1025 [ACK] Seq=1 Ack=18 Win=7504 Len=0
25	0.025486	10.0.0.1	10.0.0.5	TCP	46	2020 → 1025 [ACK] Seq=1 Ack=19 Win=7504 Len=0
26	0.025523	10.0.0.1	10.0.0.5	TCP	46	2020 → 1025 [FIN, ACK] Seq=1 Ack=19 Win=7504 Len=0
27	0.025523	10.0.0.5	10.0.0.1	TCP	46	1025 → 2020 [ACK] Seq=19 Ack=2 Win=7504 Len=0
28	0.025561	10.0.0.1	10.0.0.5	TCP	46	1025 → 21 [ACK] Seq=48 Ack=103 Win=7504 Len=0

Figure 16: Server-side PCAP: Confirms parsing of the PORT command and the subsequent RETR request for file transmission

The server-side packet trace confirms the internal command parsing logic of the application layer. In packet 10, the server receives the raw `PORT 10,0,0,1,7,228` payload. It successfully extracts the target IP address and calculates the destination port mathematically ($7 \times 256 + 228 = 2020$), acknowledging with a `200 PORT OK`. The immediate transition to the data plane, observed in packets 17 through 22, proves the server accurately mapped the parsed control command parameters to instantiate a new outgoing socket, demonstrating robust protocol compliance prior to transmitting the file.

The integrity of the transfer was verified by comparing the generated `client_received_data.txt` with the original `data.txt`. Both files contained exactly `"I am Iman Fakury."`, proving successful binary-safe transmission over the simulated network.



```
INFO: Pushing packet, packet = (Packet)tcpseg(l=11) (51 B) [Ipv4Header, version = 4, headerLength = 20 B, typeOfService = 0, totalLengthF
** Event #55  t=0.0083864  FtpNetwork.clientHost.ppp[0].ppp (Ppp, id=32)  on selfmsg pppEndTxEvent (omnetpp::cMessage, id=277)
INFO: Transmission successfully completed.
INFO (DropTailQueue)FtpNetwork.clientHost.ppp[0].queue: Pulling packet, packet = (Packet)tcpseg(l=11) (51 B) [Ipv4Header, version = 4, he
INFO: Received (inet::Packet)tcpseg(l=11) (51 B) (inet::SequenceChunk) length = 51 B from upper layer.
INFO: Transmission of (inet::Packet)tcpseg(l=11) (58 B) (inet::SequenceChunk) length = 58 B started.
** Event #56  t=0.0084328  FtpNetwork.clientHost.ppp[0].ppp (Ppp, id=32)  on selfmsg pppEndTxEvent (omnetpp::cMessage, id=277)
INFO: Transmission successfully completed.
** Running in Express mode from event #56  t=0.008850533347 ...
<!> Simulation time limit reached -- at t=60s, event #316
** Calling finish() methods of modules
INFO (Tcp)FtpNetwork.clientHost.tcp: FtpNetwork.clientHost.tcp: finishing with 2 connections open.
INFO (Tcp)FtpNetwork.serverHost.tcp: FtpNetwork.serverHost.tcp: finishing with 3 connections open.
```

Figure 17: Simulation console output: Displays successful file read/write operations and connection termination logs

The simulation console logs provide application-layer visibility into the transfer mechanics. The log entries at $t = 0.0083s$ illustrate the underlying TCP state machine pulling `tcpseg(l=11)` chunks from the transmission queue and dispatching them over the point-to-point interface. Furthermore, the termination logs explicitly confirm a clean socket lifecycle teardown. The system records finishing with precise open connection counts, ensuring no orphaned TCP sessions remain after the 226 completion code is dispatched and the application terminates.

2.2 STOR

The File Transfer Protocol utilizes the `STOR` command to facilitate file uploads from the client to the server. While the direction of the data payload is reversed compared to the download scenario, the underlying Active Mode dual-socket architecture dictates a similar connection establishment procedure.

General Operation and Connection Model

The session initiates identically to the download process, with the client securing a persistent TCP control connection on the server's Port 21 and authenticating via the `USER` command. The client then transitions into Active Mode, binding a local listening socket to port 2020 and communicating this endpoint to the server using the `PORT` command.

The key operational divergence occurs when the client issues the `STOR data.txt` command. In Active Mode, even though the client is the source of the file data, the server remains responsible for initiating the secondary TCP data connection. The server actively opens a socket and sends a TCP SYN to the client's listening port (2020). Following the three-way handshake, the client pushes the file payload—containing the string `"I am Iman Fakury."`—over this new data channel. Once the client finishes transmitting the file, it signals the closure of the data channel. The server acknowledges the successful receipt of both the data and the channel closure by sending a `226 Transfer complete` message over the persistent control plane.

Simulation Visuals

The physical packet propagation through the network demonstrates the reversed data flow. Unlike the `RETR` scenario, the data segments originate from the client node.

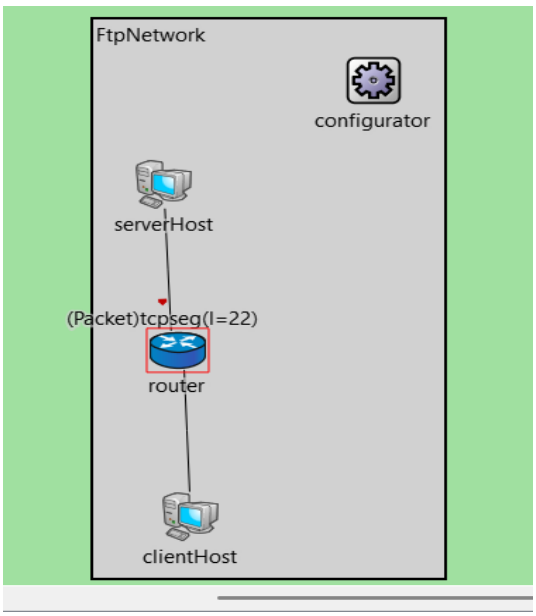


Figure 18: OMNeT++ simulation showing the tcpseg payload traversing the network from the router to the serverHost during the file upload

Wireshark Analysis

The Wireshark captures provide technical verification of the server-initiated data channel accommodating a client-driven data transfer.

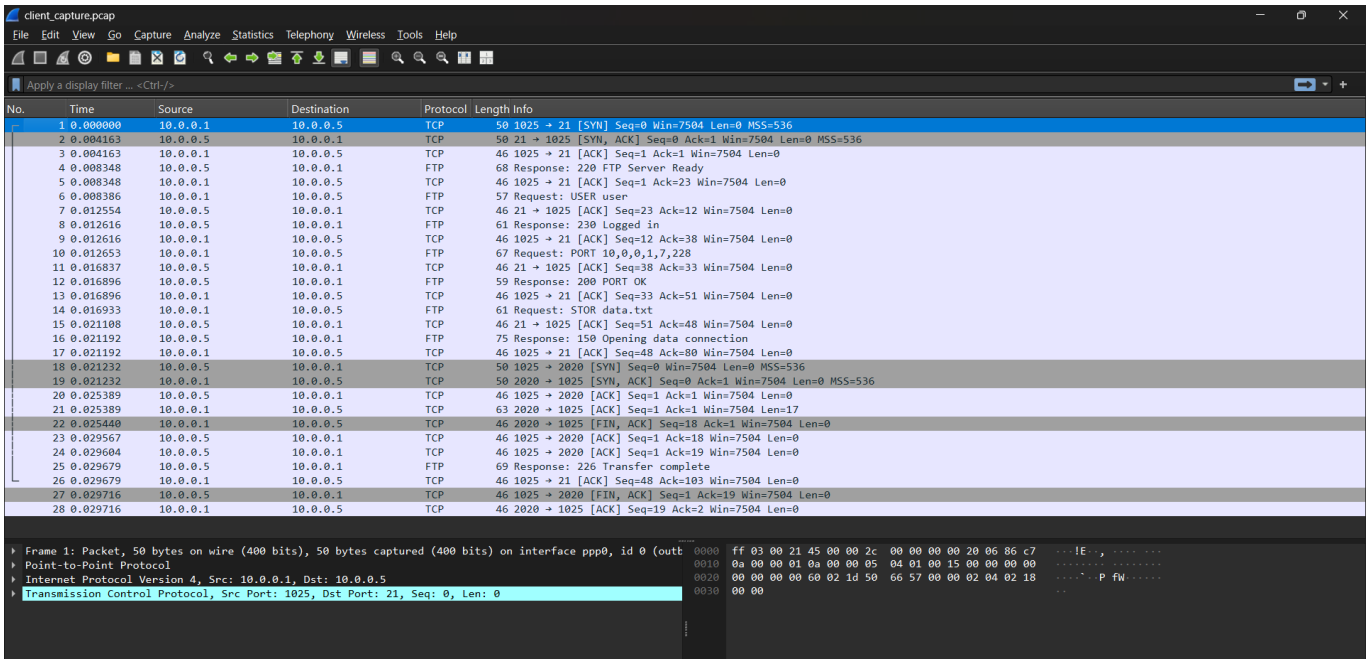


Figure 19: Client-side PCAP: Records the `STOR` command on the control plane (Packet 14) followed by the server establishing the data connection on Port 2020 (Packet 18) to receive the file

The client capture confirms precise compliance with the Active Mode specification. Following the `STOR` command dispatch, the client passively waits for the server's TCP SYN on port 2020. Once established, the client pushes the `tcpseg` payload (Packet 21) and systematically terminates the data channel (FIN, ACK sequence starting at Packet 22).

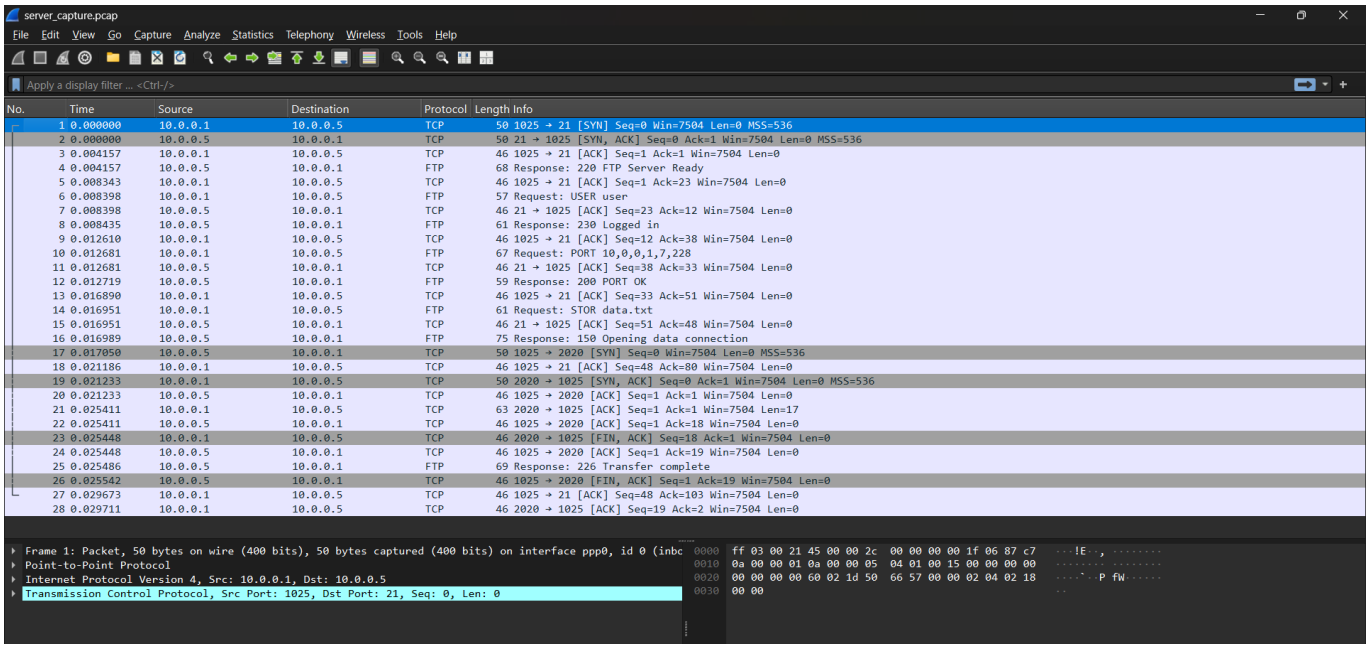


Figure 20: Server-side PCAP: Verifies the server's transition to an active connection initiator upon receiving the `STOR` command, followed by payload reception and control channel closure signaling

The server-side trace confirms proper dual-socket orchestration. Upon processing the `STOR` command (Packet 14), the server immediately transmits a `150 Opening data connection` response and initiates the required secondary socket (Packet 17). After the client pushes the file data into this socket, the server gracefully receives the payload, allows the data socket to close, and synchronizes the application state by transmitting the final `226 Transfer complete` notification on Port 21 (Packet 25).

The operational success of the protocol logic was definitively verified via the file system output. The server successfully processed the incoming data stream and generated the `server_received_data.txt` file. Inspection of this file confirmed the contents matched the original text exactly: `"I am Iman Fakury."`, proving end-to-end data integrity during the upload phase.

Bonus Part: Integration

To demonstrate the practical intersection of these application-layer protocols, a real-world integration scenario was implemented. In this topology, a single client host is tasked with uploading a file to a server, but it is only provided with the server's domain name (`fileserver.example.com`), not its raw IP address.

Differences from Previous Scenarios

In Part 1 and Part 2 of this project, the DNS and FTP protocols were simulated in isolated environments. The FTP client previously relied on OMNeT++'s internal `L3AddressResolver` to magically resolve the target server's name before the simulation even started.

This bonus scenario removes that artificial crutch and mimics a true operating system application stack. It utilizes a new star topology (`IntegratedNetwork`) centered around an `EthernetSwitch`. The client host runs **two** concurrent applications on the same node. The first application (DNS) must query the network over UDP to resolve the IP address, and only upon success can the second application (FTP) initiate its TCP connection to upload the file.

Implementation Modifications

To achieve this integration without breaking the previous simulations, backward-compatible modifications were applied to the C++ and NED files:

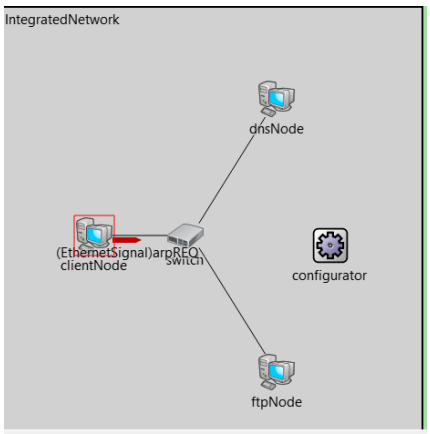
1. **FtpClient Modifications (Standby Mode):** A boolean parameter `waitForDns` was introduced. During initialization, if this flag is true, the FTP client bypasses the `connect()` call and enters a dormant state. A new public method, `connectToServer(Ipv4Address ip)`, was added to allow external triggers. Crucially, this method utilizes the `Enter_Method_Silent()` macro to allow OMNeT++ to safely switch execution contexts from the DNS application back to the FTP application.

2. **DNSClient Modifications (Trigger Logic):** The DNS client was updated with `triggerFtp` and `ftpModulePath` parameters. In the `socketDataArrived` callback, once a positive DNS answer is decoded and cached, the DNS client dynamically locates the dormant FTP module residing on the same host (e.g., `app[1]`). It then extracts the parsed `Ipv4Address` and directly invokes the FTP client's `connectToServer()` method.

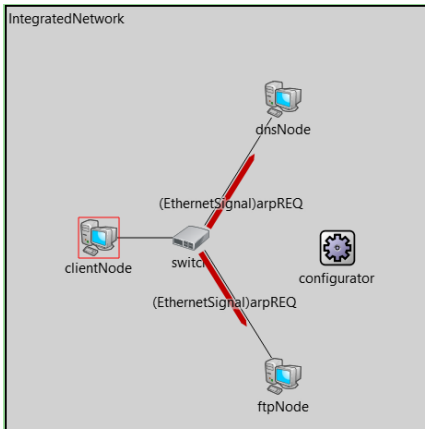
3. **Network Configuration:** The topology was updated to use `Eth100M` links connected to an Ethernet Switch, dynamically assigning IPs on the `10.0.0.x` subnet. A custom `bonus-dns.xml` database was created to map `fileserver.example.com` to the FTP node's assigned IP (`10.0.0.3`).

Step-by-Step Execution Analysis

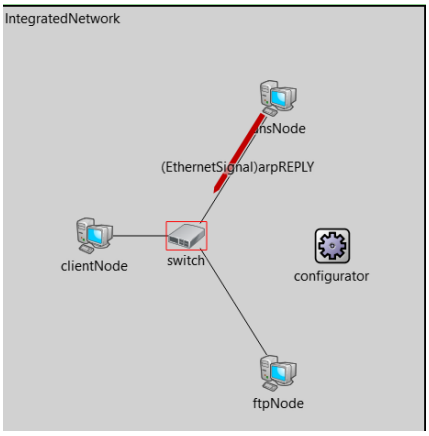
The following captures demonstrate the precise frame-by-frame execution of the network discovery and DNS resolution phase prior to the FTP handoff.



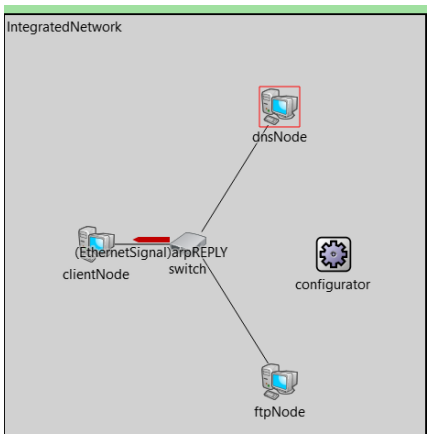
(a) Client initiates ARP Request to discover the DNS server's MAC address



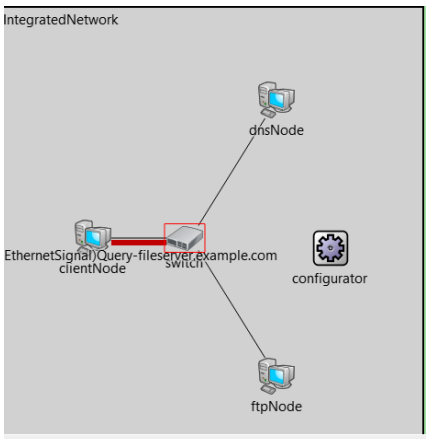
(b) The Switch broadcasts the ARP Request to all connected nodes



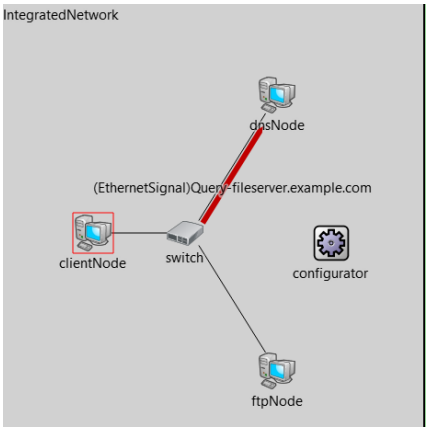
(c) The DNS Node recognizes its IP and sends a unicast ARP Reply



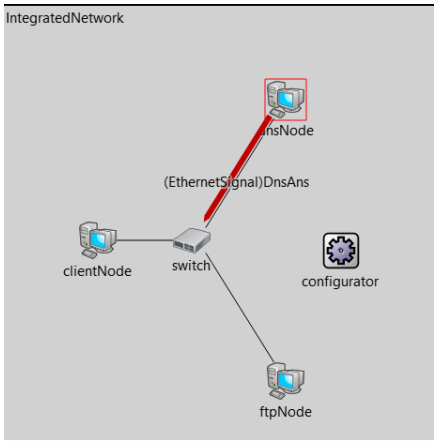
(d) The Switch forwards the ARP Reply back to the Client node



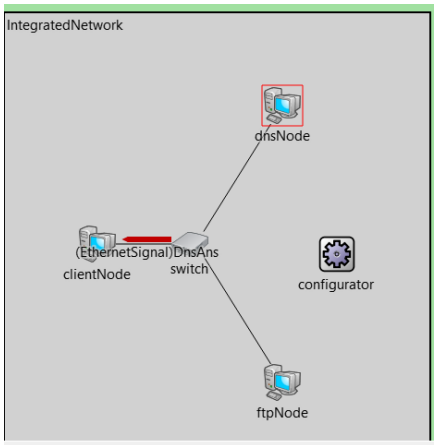
(e) Client dispatches the UDP DNS Query for `fileserver...`



(f) The Switch routes the DNS Query to the authoritative DNS Node



(g) DNS Node resolves the mapping and sends the DNS Answer



(h) Switch delivers the DNS Answer to the Client, concluding Phase 1

Figure 21: Chronological propagation of ARP and DNS packets in the integrated topology (3-3-2 Layout)

Immediately following the event depicted in Figure 18(h), the `DNSClient` parses the `10.0.0.3` payload and performs the programmatic handoff to the `FtpClient`. The FTP module wakes up and successfully initiates the Active Mode `STOR` sequence detailed in Part 2. The success of this end-to-end

integration was definitively proven by the automated generation of the `server_received_data.txt` file on the FTP node, fulfilling the ultimate goal of resolving a human-readable domain to execute a secure data transfer.